🎵 Spotify Track Popularity Prediction

End-to-End Predictive Analytics Model (KDD Process)

Author: Abhinava Akash Kotha (Roll No. 23070521075) · B.Tech CSE (AI/ML), Symbiosis Institute of Technology, Nagpur
Course component: Predictive Analytics – End-to-End Predictive Model Development (10 marks)

Notebook roadmap (mapped to the KDD / predictive-modelling stages)

#	Stage	What is done here
1	Problem identification	Business question, target variable, success metrics
2	Data selection	Load **two** Spotify datasets (tracks + artists) automatically
3	Data understanding	Structure, types, missing values, summary statistics
4	Data integration	Join the two datasets through the artist ID
5	Pre-processing	De-duplication, validity checks, outliers, missing values
6	Transformation	Feature engineering (artist reach, genres, title and date features)
7	Exploratory analysis	Distributions, correlations, trends
8	Modelling	Baseline → Linear → Ridge → Random Forest → Gradient Boosting, tuning
9	Evaluation	RMSE / MAE / R^2, residuals, error analysis, leakage ablation
10	Interpretation	Permutation importance, partial dependence, findings
11	Deployment artefacts	Saved model + data samples for GitHub

1. Problem Identification

Business question: *Can we predict how popular a Spotify track is, using its audio characteristics, release information and the reach of its artist(s)?*

Record labels, playlist curators and independent artists would benefit from a data-driven estimate of a track's popularity.

Item	Definition
Task type	Supervised learning – **regression**
Target	`popularity` – integer score 0–100 assigned by Spotify (based mainly on recent play counts)
Predictors	Audio features (danceability, energy, ...), release year, duration, explicit flag, title features, artist followers, artist genres
Metrics	RMSE (primary), MAE, R^2
Baseline	Predicting the training-set mean popularity for every track

Datasets used (2):

1. `tracks.csv` – ~600k tracks with audio features and popularity.
2. `artists.csv` – ~1.1M artists with followers, genres and artist popularity.

> ⚠️ **Data-leakage note.** Spotify defines *artist popularity* as a function of the popularity of the artist's tracks — which includes the very track we try to predict. Using it would leak the target. We therefore treat it separately and quantify its effect in an ablation study (Section 9.3).

2. Data Selection & Loading

The datasets are downloaded **programmatically** from Kaggle (`yamaerenay/spotify-dataset-19212020-600k-tracks`) via `kagglehub` — no manual download is needed. If Kaggle is unreachable, the notebook falls back to the sample CSVs stored in this repository's `data/` folder.

```
%pip install -q kagglehub
```

```
import os, re, glob, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.base import clone
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance, PartialDependenceDisplay
from sklearn.linear_model import LinearRegression, Ridge
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold, RandomizedSearchCV, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
sns.set_theme(style="whitegrid", context="notebook")
pd.set_option("display.max_columns", 50)
os.makedirs("data", exist_ok=True)
os.makedirs("models", exist_ok=True)
```

```
KAGGLE_SLUG = "yamaerenay/spotify-dataset-19212020-600k-tracks"

def load_from_kaggle():
    import kagglehub
    path = kagglehub.dataset_download(KAGGLE_SLUG)
    tracks_path = glob.glob(os.path.join(path, "**", "tracks.csv"), recursive=True)[0]
    artists_path = glob.glob(os.path.join(path, "**", "artists.csv"), recursive=True)[0]
    return pd.read_csv(tracks_path), pd.read_csv(artists_path)

def load_from_local():
    return (pd.read_csv("data/tracks_sample.csv"),
            pd.read_csv("data/artists_sample.csv"))

try:
    tracks_raw, artists_raw = load_from_kaggle()
    DATA_SOURCE = "kaggle"
except Exception as e:
    print(f"Kaggle download failed ({type(e).__name__}); using local sample files in data/ instead.")
    tracks_raw, artists_raw = load_from_local()
    DATA_SOURCE = "local-sample"

print(f"Data source : {DATA_SOURCE}")
print(f"tracks.csv  : {tracks_raw.shape[0]:,} rows x {tracks_raw.shape[1]} cols")
print(f"artists.csv : {artists_raw.shape[0]:,} rows x {artists_raw.shape[1]} cols")
```

```
Downloading to /root/.cache/kagglehub/datasets/yamaerenay/spotify-dataset-19212020-600k-tracks/1.archive...
100%|██████████| 193M/193M [00:01<00:00, 114MB/s]Extracting files...

Data source : kaggle
tracks.csv  : 586,672 rows x 20 cols
artists.csv : 1,162,095 rows x 5 cols
```

3. Data Understanding

```
display(tracks_raw.head())
display(artists_raw.head())
```

	id	name	popularity	duration_ms	explicit	artists	id_artists	release_d
0	35iwgR4jXetI318WEWsa1Q	Carve	6	126903	0	['Uli']	['45tIt06XoI0Iio4LBEVpls']	1922-02
1	021ht4sdgPcrDgSk7JTbKY	Capítulo 2.16 - Banquero Anarquista	0	98200	0	['Fernando Pessoa']	['14jtPCOoNZwquk5wd9DxrY']	1922-06
2	07A5yehtSnoedViJAZkNnc	Vivo para Quererte - Remasterizado	0	181640	0	['Ignacio Corsini']	['5LiOoJbxVSAMkBS2fUm3X2']	1922-03
3	08FmqUhxtyLTn6pAh6bk45	El Prisionero - Remasterizado	0	176907	0	['Ignacio Corsini']	['5LiOoJbxVSAMkBS2fUm3X2']	1922-03
4	08y9GfoqCWfOGsKdwojr5e	Lady of the Evening	0	163080	0	['Dick Haymes']	['3BiJGZsyX9sJchTqcSA7Su']	1

	id	followers	genres	name	popularity
0	0DheY5irMjBUeLybbCUEZ2	0.0	[]	Armid & Amir Zare Pashai feat. Sara Rouzbehani	0
1	0DlhY15l3wsrnlfGio2bjU	5.0	[]	ปู่นา ภาวิณี	0
2	0DmRESX2JknGPQyO15yxg7	0.0	[]	Sadaa	0
3	0DmhnbHjm1qw6NCYPeZNgJ	0.0	[]	Tra'gruda	0
4	0Dn11fWM7vHQ3rinvWEl4E	2.0	[]	Ioannis Panoutsopoulos	0

```
print("=== tracks: dtypes & missing values ===")
info = pd.DataFrame({"dtype": tracks_raw.dtypes.astype(str),
```

```
                    "missing": tracks_raw.isna().sum(),
                    "missing_%": (tracks_raw.isna().mean() * 100).round(2),
                    "n_unique": tracks_raw.nunique()})
display(info)

print("=== artists: dtypes & missing values ===")
info_a = pd.DataFrame({"dtype": artists_raw.dtypes.astype(str),
                       "missing": artists_raw.isna().sum(),
                       "missing_%": (artists_raw.isna().mean() * 100).round(2),
                       "n_unique": artists_raw.nunique()})
display(info_a)
```

```
=== tracks: dtypes & missing values ===
```

	dtype	missing	missing_%	n_unique
id	object	0	0.00	586672
name	object	71	0.01	446474
popularity	int64	0	0.00	101
duration_ms	int64	0	0.00	123122
explicit	int64	0	0.00	2
artists	object	0	0.00	114030
id_artists	object	0	0.00	115062
release_date	object	0	0.00	19700
danceability	float64	0	0.00	1285
energy	float64	0	0.00	2571
key	int64	0	0.00	12
loudness	float64	0	0.00	29196
mode	int64	0	0.00	2
speechiness	float64	0	0.00	1655
acousticness	float64	0	0.00	5217
instrumentalness	float64	0	0.00	5402
liveness	float64	0	0.00	1782
valence	float64	0	0.00	1805
tempo	float64	0	0.00	122706
time_signature	int64	0	0.00	5

```
=== artists: dtypes & missing values ===
```

	dtype	missing	missing_%	n_unique
id	object	0	0.0	1162095
followers	float64	11	0.0	51998
genres	object	0	0.0	49155
name	object	3	0.0	1134429
popularity	int64	0	0.0	99

Next steps: Generate code with `info` | New interactive sheet | Generate code with `info_a` | New interactive sheet

```
display(tracks_raw.describe().T.round(3))
display(artists_raw.describe().T.round(3))
```

	count	mean	std	min	25%	50%	75%	max
popularity	586672.0	27.570	18.371	0.0	13.000	27.000	41.000	100.000
duration_ms	586672.0	230051.167	126526.087	3344.0	175093.000	214893.000	263867.000	5621218.000
explicit	586672.0	0.044	0.205	0.0	0.000	0.000	0.000	1.000
danceability	586672.0	0.564	0.166	0.0	0.453	0.577	0.686	0.991
energy	586672.0	0.542	0.252	0.0	0.343	0.549	0.748	1.000
key	586672.0	5.222	3.519	0.0	2.000	5.000	8.000	11.000
loudness	586672.0	-10.206	5.089	-60.0	-12.891	-9.243	-6.482	5.376
mode	586672.0	0.659	0.474	0.0	0.000	1.000	1.000	1.000
speechiness	586672.0	0.105	0.180	0.0	0.034	0.044	0.076	0.971
acousticness	586672.0	0.450	0.349	0.0	0.097	0.422	0.785	0.996
instrumentalness	586672.0	0.113	0.267	0.0	0.000	0.000	0.010	1.000
liveness	586672.0	0.214	0.184	0.0	0.098	0.139	0.278	1.000
valence	586672.0	0.552	0.258	0.0	0.346	0.564	0.769	1.000
tempo	586672.0	118.465	29.764	0.0	95.600	117.384	136.321	246.381
time_signature	586672.0	3.873	0.473	0.0	4.000	4.000	4.000	5.000

	count	mean	std	min	25%	50%	75%	max
followers	1162084.0	10220.704	254399.493	0.0	10.0	57.0	417.0	78900234.0
popularity	1162095.0	8.796	13.558	0.0	0.0	2.0	13.0	100.0

Observations

- `tracks` holds track-level attributes; the link to `artists` is the column `id_artists` (a *string-encoded list* of artist IDs).
- `artists` holds `followers`, `genres` (also a string-encoded list) and `popularity` of the artist.
- Several columns need attention: string-encoded lists, mixed-format `release_date`, a few zero values in `tempo` / `time_signature`, and many artists without genres.

4. Data Integration (joining the two datasets)

A track may have several artists, so a plain one-to-one merge is not possible. For each track we:

1. extract the list of artist IDs from `id_artists`,
2. look up each artist in `artists.csv`,
3. aggregate across the track's artists (mean / max followers, mean / max artist popularity),
4. take the **primary (first) artist's genres**.

```
def parse_ids(s):
    return re.findall(r"'([^']+)'", s) if isinstance(s, str) else []

artists = artists_raw.drop_duplicates(subset="id").copy()
followers_map = dict(zip(artists["id"], artists["followers"]))
apop_map      = dict(zip(artists["id"], artists["popularity"]))
genres_map    = dict(zip(artists["id"], artists["genres"].fillna("[]")))

tracks = tracks_raw.copy()
id_lists = tracks["id_artists"].map(parse_ids)

def agg_stat(ids, mapping, func):
    vals = [mapping[i] for i in ids if i in mapping and pd.notna(mapping[i])]
    return func(vals) if vals else np.nan

tracks["n_artists"]        = id_lists.map(len).clip(lower=1)
tracks["followers_mean"]   = [agg_stat(i, followers_map, np.mean) for i in id_lists]
tracks["followers_max"]    = [agg_stat(i, followers_map, np.max)  for i in id_lists]
tracks["artist_pop_mean"]  = [agg_stat(i, apop_map, np.mean)      for i in id_lists]
tracks["artist_pop_max"]   = [agg_stat(i, apop_map, np.max)       for i in id_lists]
tracks["primary_genres"]   = [genres_map.get(i[0], "[]") if i else "[]" for i in id_lists]

matched = tracks["followers_mean"].notna().mean() * 100
print(f"Tracks successfully linked to at least one artist record: {matched:.1f}%")
tracks[["name", "artists", "n_artists", "followers_mean", "artist_pop_mean", "primary_genres"]].head()
```

```
Tracks successfully linked to at least one artist record: 98.2%
```

	name	artists	n_artists	followers_mean	artist_pop_mean	primary_genres
0	Carve	['Uli']	1	91.0	4.0	[]
1	Capítulo 2.16 - Banquero Anarquista	['Fernando Pessoa']	1	3.0	0.0	[]
2	Vivo para Quererte - Remasterizado	['Ignacio Corsini']	1	3528.0	23.0	['tango', 'vintage tango']
3	El Prisionero -	['Ignacio	1	3528.0	23.0	['tango', 'vintage tango']

5. Data Pre-processing

Steps: remove duplicates → drop rows without a title → validate/parse the release year → treat impossible audio values → cap extreme durations.
Remaining missing values are imputed **inside the modelling pipeline** (fit on training data only) to avoid information leakage.

```
n0 = len(tracks)
tracks = tracks.drop_duplicates(subset="id")
tracks = tracks.dropna(subset=["name"])
print(f"Removed {n0 - len(tracks):,} duplicate / unnamed rows")

# release year: the date is stored as YYYY, YYYY-MM or YYYY-MM-DD
tracks["release_year"] = pd.to_numeric(tracks["release_date"].astype(str).str[:4], errors="coerce")
tracks = tracks[tracks["release_year"].between(1900, 2021)]

# physically impossible values -> missing
tracks.loc[tracks["tempo"] <= 0, "tempo"] = np.nan
tracks.loc[tracks["time_signature"] <= 0, "time_signature"] = np.nan

# duration in minutes, capped at the 99.9th percentile (a few podcast-like outliers)
dur_cap = tracks["duration_ms"].quantile(0.999)
tracks["duration_min"] = tracks["duration_ms"].clip(upper=dur_cap) / 60000

print(f"Rows after cleaning: {len(tracks):,}")
print("\nMissing values after cleaning (features only):")
miss = tracks[["tempo", "time_signature", "followers_mean", "artist_pop_mean"]].isna().sum()
display(miss.to_frame("missing"))
```

```
Removed 71 duplicate / unnamed rows
Rows after cleaning: 586,601

Missing values after cleaning (features only):
```

	missing
tempo	328
time_signature	337
followers_mean	10475
artist_pop_mean	10475

6. Data Transformation (Feature Engineering)

Feature group	Features
Audio	danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, time_signature
Track metadata	duration_min, explicit, release_year, track_age, n_artists
Title	name_len, name_words, has_feat
Artist reach	log followers (mean / max), has_artist_info
Artist genres	genre_count + 18 broad-genre flags (pop, rock, hip hop, ...)
Leaky (ablation only)	artist_pop_mean, artist_pop_max

Follower counts are extremely right-skewed, so a `log1p` transform is applied.

```
REFERENCE_YEAR = 2021
tracks["track_age"] = REFERENCE_YEAR - tracks["release_year"]

# title features
tracks["name_len"]   = tracks["name"].str.len()
tracks["name_words"] = tracks["name"].str.split().str.len()
tracks["has_feat"]   = tracks["name"].str.contains(r"feat|ft\.|with ", case=False, regex=True).astype(int)

# artist reach (log transform for skewed counts)
tracks["log_followers_mean"] = np.log1p(tracks["followers_mean"])
```

```
tracks["log_followers_max"]  = np.log1p(tracks["followers_max"])
tracks["has_artist_info"]    = tracks["followers_mean"].notna().astype(int)

# genre features from the primary artist
tracks["genre_count"] = tracks["primary_genres"].str.count("'") // 2
GENRE_KEYWORDS = {
    "pop": r"pop", "rock": r"rock", "hiphop": r"hip hop|rap|trap", "latin": r"latin|reggaeton|salsa|urbano",
    "indie": r"indie", "electronic": r"electro|edm|house|techno|trance|dubstep",
    "jazz": r"jazz", "classical": r"classical|orchestra|baroque|opera", "metal": r"metal",
    "country": r"country", "rnb": r"r&b|soul|funk", "folk": r"folk|singer-songwriter",
    "dance": r"dance", "reggae": r"reggae|ska", "blues": r"blues", "punk": r"punk",
    "soundtrack": r"soundtrack|score|movie|film", "kids": r"kids|children|lullaby",
}
for g, pattern in GENRE_KEYWORDS.items():
    tracks[f"g_{g}"] = tracks["primary_genres"].str.contains(pattern, case=False, regex=True).astype(int)

AUDIO = ["danceability", "energy", "key", "loudness", "mode", "speechiness", "acousticness",
         "instrumentalness", "liveness", "valence", "tempo", "time_signature"]
META  = ["duration_min", "explicit", "release_year", "track_age", "n_artists",
         "name_len", "name_words", "has_feat"]
REACH = ["log_followers_mean", "log_followers_max", "has_artist_info"]
GENRE = ["genre_count"] + [f"g_{g}" for g in GENRE_KEYWORDS]
LEAKY = ["artist_pop_mean", "artist_pop_max"]
TARGET = "popularity"

# release_year and track_age are perfectly collinear -> keep only track_age for linear models' sake
META = [c for c in META if c != "release_year"]

data = tracks[["id", "name", "artists", TARGET] + AUDIO + META + REACH + GENRE + LEAKY].copy()
print("Final modelling table:", data.shape)
data.head()
```

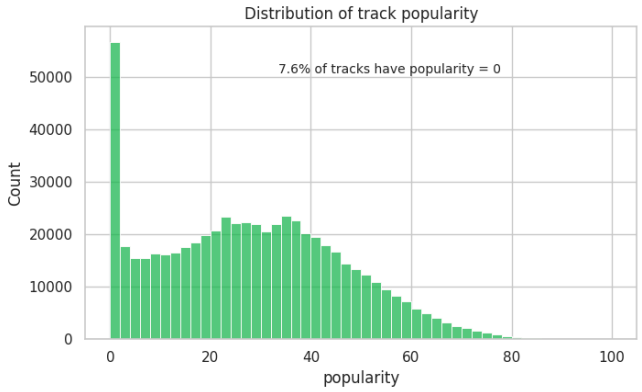
```
Final modelling table: (586601, 47)
```

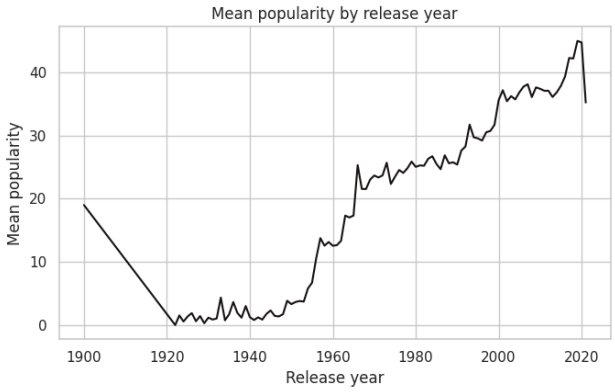
	id	name	artists	popularity	danceability	energy	key	loudness	mode	speechiness	acou
0	35iwgR4jXetI318WEWsa1Q	Carve	['Uli']	6	0.645	0.4450	0	-13.338	1	0.4510	
1	021ht4sdgPcrDgSk7JTbKY	Capítulo 2.16 - Banquero Anarquista	['Fernando Pessoa']	0	0.695	0.2630	0	-22.136	1	0.9570	
2	07A5yehtSnoedViJAZkNnc	Vivo para Quererte - Remasterizado	['Ignacio Corsini']	0	0.434	0.1770	1	-21.180	1	0.0512	
3	08FmqUhxtyLTn6pAh6bk45	El Prisionero - Remasterizado	['Ignacio Corsini']	0	0.321	0.0946	7	-27.961	1	0.0504	
4	08y9GfoqCWfOGsKdwojr5e	Lady of the Evening	['Dick Haymes']	0	0.402	0.1580	3	-16.900	0	0.0390	

7. Exploratory Data Analysis (EDA)

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
sns.histplot(data[TARGET], bins=50, ax=axes[0], color="#1DB954")
axes[0].set_title("Distribution of track popularity")
zero_share = (data[TARGET] == 0).mean() * 100
axes[0].annotate(f"{zero_share:.1f}% of tracks have popularity = 0", xy=(0.35, 0.85),
                 xycoords="axes fraction", fontsize=10)

yearly = data.groupby(2021 - data["track_age"])[TARGET].mean()
yearly.plot(ax=axes[1], color="#191414")
axes[1].set_title("Mean popularity by release year")
axes[1].set_xlabel("Release year"); axes[1].set_ylabel("Mean popularity")
plt.tight_layout(); plt.show()
```

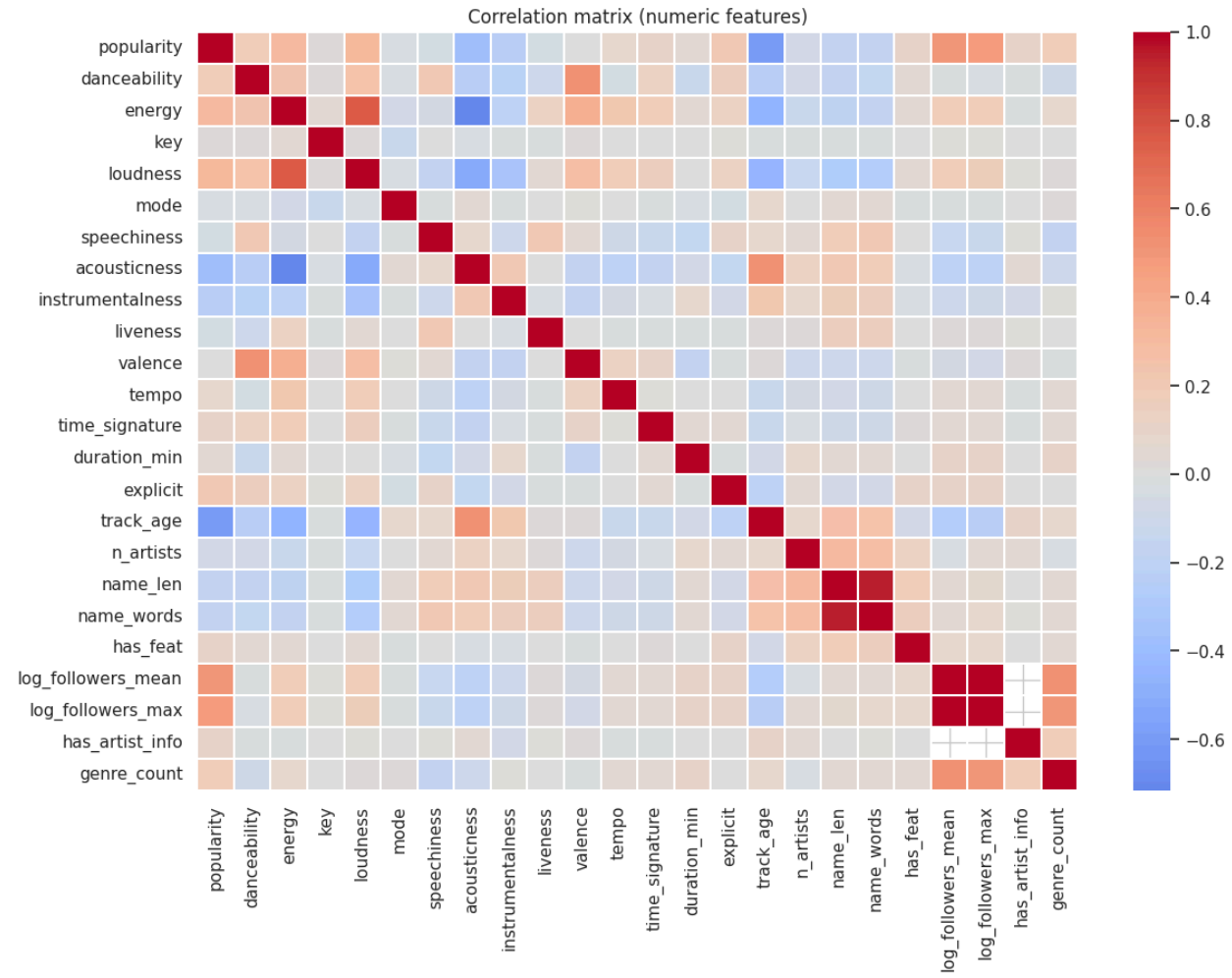




Insight (check against your plots): Spotify's popularity metric rewards *recent* plays, so newer tracks are expected to score higher on average, and a spike at 0 is expected for tracks nobody is currently streaming.

```
plt.figure(figsize=(13, 9))
corr = data[[TARGET] + AUDIO + META + REACH + ["genre_count"]].corr()
sns.heatmap(corr, cmap="coolwarm", center=0, annot=False, linewidths=0.2)
plt.title("Correlation matrix (numeric features)")
plt.show()

print("Correlation with popularity (top 12 by absolute value):")
display(corr[TARGET].drop(TARGET).sort_values(key=abs, ascending=False).head(12).round(3).to_frame("corr"))
```



Correlation with popularity (top 12 by absolute value):

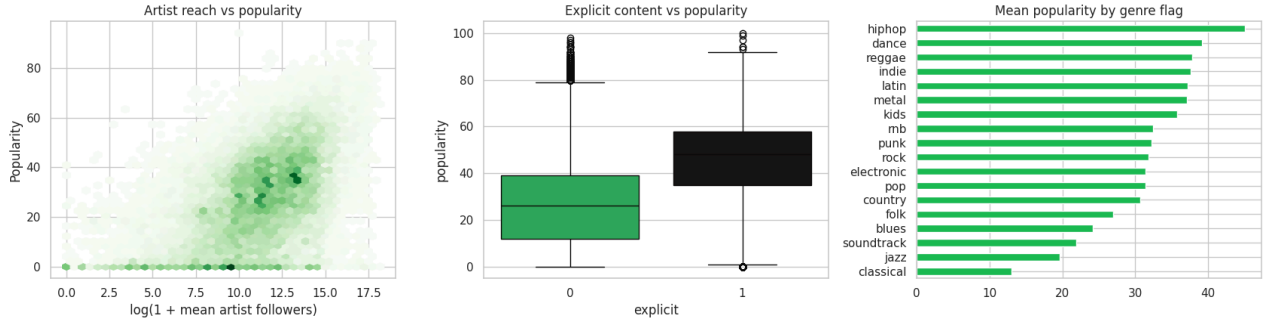
	corr
track_age	-0.591
log_followers_mean	0.494
log_followers_max	0.482
acousticness	-0.371
loudness	0.327
energy	0.302
instrumentalness	-0.236
explicit	0.212
danceability	0.187
name_len	-0.180
genre_count	0.173
name_words	-0.156

```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))

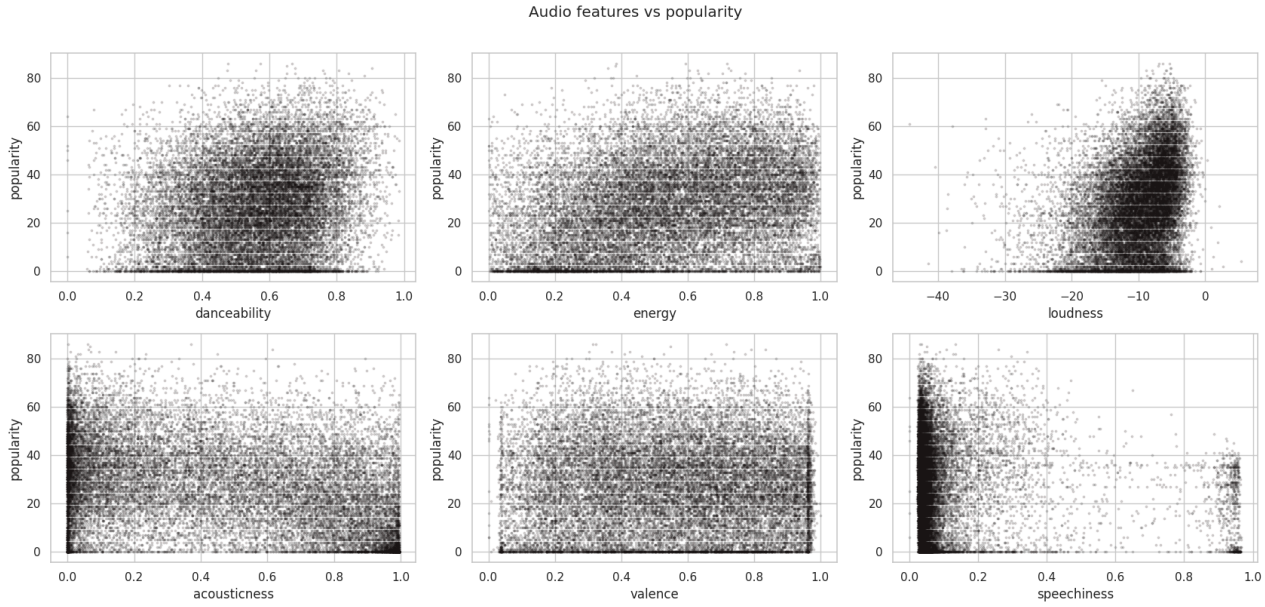
has_reach = data.dropna(subset=["log_followers_mean"])
samp = has_reach.sample(min(30000, len(has_reach)), random_state=RANDOM_STATE)
axes[0].hexbin(samp["log_followers_mean"], samp[TARGET], gridsize=40, cmap="Greens", mincnt=1)
axes[0].set_xlabel("log(1 + mean artist followers)"); axes[0].set_ylabel("Popularity")
axes[0].set_title("Artist reach vs popularity")

sns.boxplot(x="explicit", y=TARGET, data=data, ax=axes[1], palette=["#1DB954", "#191414"])
axes[1].set_title("Explicit content vs popularity")
```

```
genre_means = {g: data.loc[data[f"g_{g}"] == 1, TARGET].mean() for g in GENRE_KEYWORDS}
pd.Series(genre_means).sort_values().plot.barh(ax=axes[2], color="#1DB954")
axes[2].set_title("Mean popularity by genre flag")
plt.tight_layout(); plt.show()
```




```
fig, axes = plt.subplots(2, 3, figsize=(17, 8))
for ax, col in zip(axes.ravel(), ["danceability", "energy", "loudness", "acousticness", "valence", "speechiness"]):
    s = data.sample(min(20000, len(data)), random_state=RANDOM_STATE)
    ax.scatter(s[col], s[TARGET], s=3, alpha=0.15, color="#191414")
    ax.set_xlabel(col); ax.set_ylabel("popularity")
plt.suptitle("Audio features vs popularity", y=1.01)
plt.tight_layout(); plt.show()
```



EDA takeaways (confirm against your own plots and correlation table)

- Artist reach (followers) and release recency are expected to show the clearest relationship with popularity.
- Individual audio features are typically only **weakly** correlated with popularity on their own, so non-linear, interaction-capable models (tree ensembles) are worth testing against the linear benchmarks.

8. Modelling

8.1 Train / test split

To keep training time reasonable on free Colab, modelling uses a random sample (`SAMPLE_N`) of the cleaned data. An 80/20 hold-out split is created; the test set is **never** touched until final evaluation.

```
SAMPLE_N = 150_000
model_df = data.sample(n=min(SAMPLE_N, len(data)), random_state=RANDOM_STATE).reset_index(drop=True)

FEATURES_A = AUDIO + META + GENRE                 # content-only (no artist info)
FEATURES_B = AUDIO + META + GENRE + REACH         # + artist reach  <- main model
FEATURES_C = FEATURES_B + LEAKY                   # + leaky artist popularity (ablation only)

X_train, X_test, y_train, y_test = train_test_split(
    model_df, model_df[TARGET], test_size=0.20, random_state=RANDOM_STATE)

print(f"Train: {len(X_train):,}   Test: {len(X_test):,}")
print(f"Features in main model (B): {len(FEATURES_B)}")
```

```
Train: 120,000   Test: 30,000
Features in main model (B): 41
```

8.2 Candidate models

Model	Why
Dummy (mean)	Baseline – any useful model must beat it
Linear Regression	Simple, interpretable benchmark
Ridge Regression	Regularised linear model (handles collinearity)
Random Forest	Non-linear, bagged trees
Histogram Gradient Boosting	Boosted trees, fast, handles missing values natively

Linear models use median imputation + standardisation inside a `Pipeline`; tree models use median imputation only.

```
def make_pipeline(estimator, scale=False):
    steps = [("impute", SimpleImputer(strategy="median"))]
    if scale:
        steps.append(("scale", StandardScaler()))
    steps.append(("model", estimator))
    return Pipeline(steps)

models = {
    "Baseline (mean)":     make_pipeline(DummyRegressor(strategy="mean")),
    "Linear Regression":   make_pipeline(LinearRegression(), scale=True),
    "Ridge Regression":    make_pipeline(Ridge(alpha=10.0), scale=True),
    "Random Forest":       make_pipeline(RandomForestRegressor(n_estimators=120, max_depth=14,
                                         min_samples_leaf=5, n_jobs=-1, random_state=RANDOM_STATE)),
    "Hist Gradient Boosting": HistGradientBoostingRegressor(max_iter=300, learning_rate=0.08,
                                         early_stopping=True, random_state=RANDOM_STATE),
}

def rmse(y, p): return float(np.sqrt(mean_squared_error(y, p)))

# 3-fold cross-validation on a 40k subsample of the training set (keeps runtime low)
cv_idx = X_train.sample(n=min(40_000, len(X_train)), random_state=RANDOM_STATE).index
kf = KFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)

cv_rows = []
for name, mdl in models.items():
    scores = cross_val_score(mdl, X_train.loc[cv_idx, FEATURES_B], y_train.loc[cv_idx],
                             scoring="neg_root_mean_squared_error", cv=kf, n_jobs=1)
    cv_rows.append({"Model": name, "CV RMSE (mean)": -scores.mean(), "CV RMSE (std)": scores.std()})
    print(f"{name:25s} CV RMSE = {-scores.mean():6.3f} ± {scores.std():.3f}")

cv_table = pd.DataFrame(cv_rows).set_index("Model").round(3)
cv_table
```

```
Baseline (mean)           CV RMSE = 18.272 ± 0.044
Linear Regression         CV RMSE = 12.536 ± 0.013
Ridge Regression          CV RMSE = 12.536 ± 0.014
Random Forest             CV RMSE = 10.951 ± 0.019
Hist Gradient Boosting    CV RMSE = 10.921 ± 0.031
```

Model	CV RMSE (mean)	CV RMSE (std)
Baseline (mean)	18.272	0.044
Linear Regression	12.536	0.013
Ridge Regression	12.536	0.014
Random Forest	10.951	0.019
Hist Gradient Boosting	10.921	0.031

Next steps: Generate code with `cv_table` | New interactive sheet

8.3 Hyper-parameter tuning (Gradient Boosting)

`RandomizedSearchCV` explores 10 random configurations with 3-fold CV on a training subsample; the best configuration is then refit on the **full** training set.

```
param_dist = {
    "learning_rate":     [0.03, 0.05, 0.08, 0.12],
    "max_iter":          [200, 400, 600],
    "max_depth":         [None, 6, 10],
    "max_leaf_nodes":    [31, 63, 127],
    "min_samples_leaf":  [20, 50, 100],
    "l2_regularization": [0.0, 1.0, 10.0],
}
search = RandomizedSearchCV(
    HistGradientBoostingRegressor(early_stopping=True, random_state=RANDOM_STATE),
    param_distributions=param_dist, n_iter=10, cv=3,
    scoring="neg_root_mean_squared_error", random_state=RANDOM_STATE, n_jobs=1, verbose=1)
search.fit(X_train.loc[cv_idx, FEATURES_B], y_train.loc[cv_idx])

print("Best params :", search.best_params_)
print(f"Best CV RMSE: {-search.best_score_:.3f}")

best_hgb = HistGradientBoostingRegressor(early_stopping=True, random_state=RANDOM_STATE,
                                         **search.best_params_)
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Best params : {'min_samples_leaf': 50, 'max_leaf_nodes': 127, 'max_iter': 400, 'max_depth': 10, 'learning_rate': 0.03, 'l2_r
Best CV RMSE: 10.867
```

9. Evaluation (on the untouched test set)

```
final_models = {k: v for k, v in models.items() if k != "Hist Gradient Boosting"}
final_models["Hist Gradient Boosting (tuned)"] = best_hgb

results, preds = [], {}
for name, mdl in final_models.items():
    mdl.fit(X_train[FEATURES_B], y_train)
    p = np.clip(mdl.predict(X_test[FEATURES_B]), 0, 100)
    preds[name] = p
    results.append({"Model": name, "RMSE": rmse(y_test, p),
                    "MAE": mean_absolute_error(y_test, p), "R²": r2_score(y_test, p)})

results_df = pd.DataFrame(results).set_index("Model").round(3).sort_values("RMSE")
display(results_df)

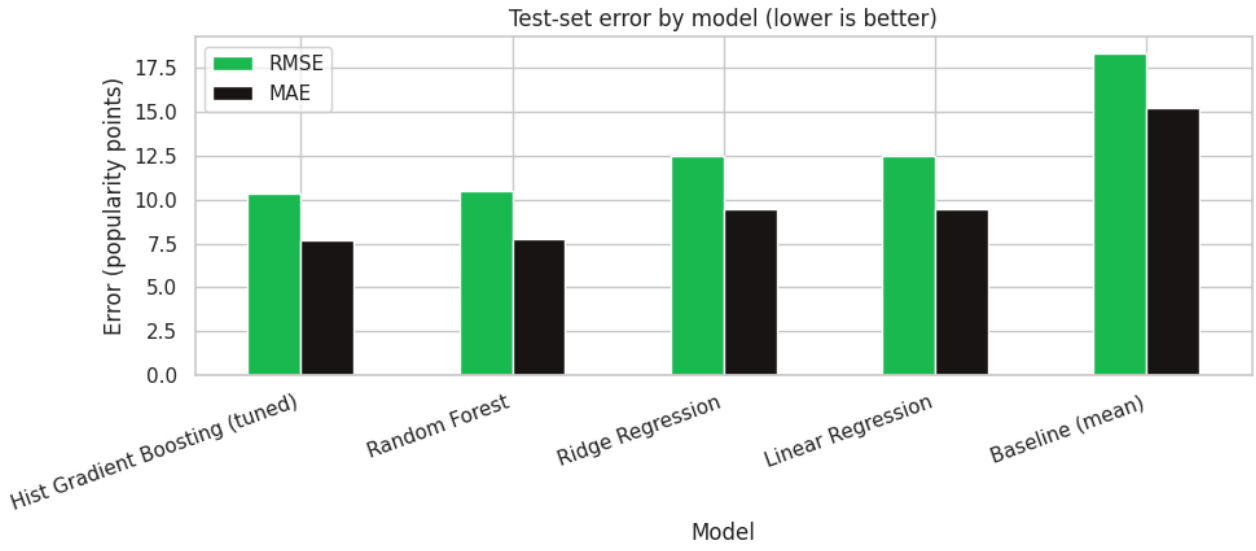
best_name = results_df.index[0]
print(f"\n⏳ Best model on test set: {best_name}")
```

Model	RMSE	MAE	R²
Hist Gradient Boosting (tuned)	10.396	7.696	0.680
Random Forest	10.513	7.764	0.672
Ridge Regression	12.468	9.485	0.539
Linear Regression	12.468	9.484	0.539
Baseline (mean)	18.367	15.273	-0.000

Best model on test set: Hist Gradient Boosting (tuned)

Next steps: Generate code with `results_df` New interactive sheet

```
results_df[["RMSE", "MAE"]].plot.bar(figsize=(10, 4.5), color=["#1DB954", "#191414"])
plt.title("Test-set error by model (lower is better)")
plt.ylabel("Error (popularity points)"); plt.xticks(rotation=20, ha="right")
plt.tight_layout(); plt.show()
```



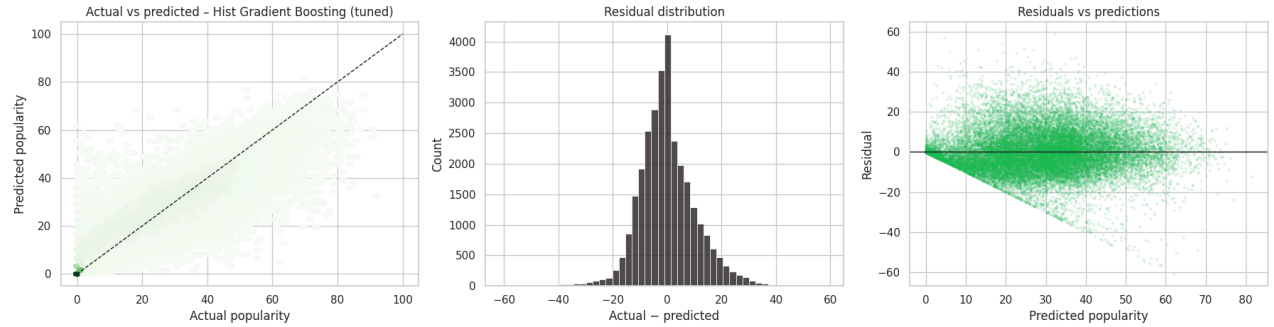
9.1 Actual vs predicted and residual analysis

```
best_pred = preds[best_name]
resid = y_test.values - best_pred

fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))
axes[0].hexbin(y_test, best_pred, gridsize=45, cmap="Greens", mincnt=1)
axes[0].plot([0, 100], [0, 100], "k--", lw=1)
axes[0].set_xlabel("Actual popularity"); axes[0].set_ylabel("Predicted popularity")
axes[0].set_title(f"Actual vs predicted – {best_name}")

sns.histplot(resid, bins=50, ax=axes[1], color="#191414")
axes[1].set_title("Residual distribution"); axes[1].set_xlabel("Actual − predicted")

axes[2].scatter(best_pred, resid, s=3, alpha=0.1, color="#1DB954")
axes[2].axhline(0, color="k", lw=1)
axes[2].set_xlabel("Predicted popularity"); axes[2].set_ylabel("Residual")
axes[2].set_title("Residuals vs predictions")
plt.tight_layout(); plt.show()
```



9.2 Error analysis by popularity band

```
bands = pd.cut(y_test, bins=[-1, 0, 20, 40, 60, 80, 100],
               labels=["0", "1-20", "21-40", "41-60", "61-80", "81-100"])
err = pd.DataFrame({"band": bands.values, "abs_err": np.abs(resid), "resid": resid})
band_tbl = err.groupby("band").agg(n=("abs_err", "size"), MAE=("abs_err", "mean"), mean_residual=("resid", "mean")).round(
display(band_tbl)
```

band	n	MAE	mean_residual
0	2262	4.19	-4.19
1-20	8939	7.71	-6.53
21-40	11185	6.41	0.12
41-60	6348	9.54	7.60
61-80	1225	15.67	15.29
81-100	41	26.00	26.00

Next steps: Generate code with band_tbl | New interactive sheet

Check the table above: regression models commonly regress towards the mean, i.e. **over-predict** tracks with popularity 0 and **under-predict** very popular hits (positive mean residual in the top bands). This is typical for a bounded, skewed target.

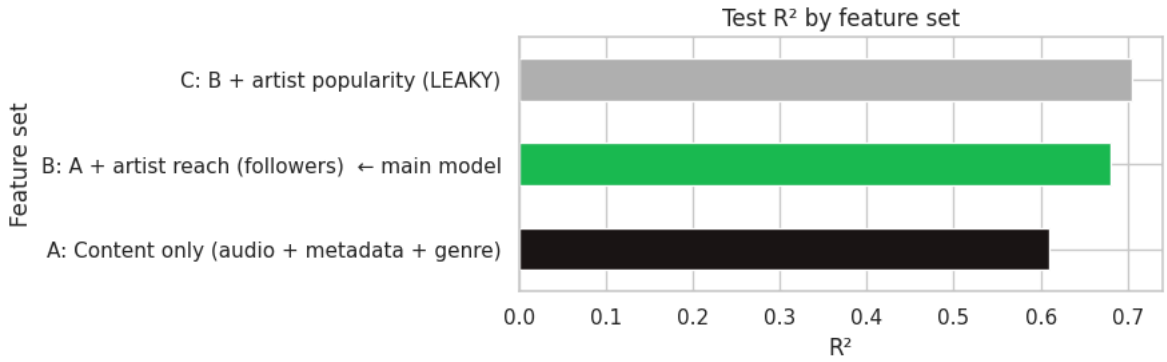
9.3 Ablation study – what does each information source add?

We refit the tuned gradient-boosting model on three feature sets to quantify (a) the value of artist information and (b) how much a **leaky** feature would inflate performance.

```
ablation = []
for label, feats in [("A: Content only (audio + metadata + genre)", FEATURES_A),
                     ("B: A + artist reach (followers)  ← main model", FEATURES_B),
                     ("C: B + artist popularity (LEAKY)", FEATURES_C)]:
    m = clone(best_hgb).fit(X_train[feats], y_train)
    p = np.clip(m.predict(X_test[feats]), 0, 100)
    ablation.append({"Feature set": label, "n_features": len(feats),
                     "RMSE": rmse(y_test, p), "MAE": mean_absolute_error(y_test, p), "R²": r2_score(y_test, p)})
ablation_df = pd.DataFrame(ablation).set_index("Feature set").round(3)
display(ablation_df)

ablation_df["R²"].plot.barh(figsize=(9, 3), color=["#191414", "#1DB954", "#B3B3B3"])
plt.title("Test R² by feature set"); plt.xlabel("R²"); plt.tight_layout(); plt.show()
```

Feature set	n_features	RMSE	MAE	R²
A: Content only (audio + metadata + genre)	38	11.465	8.515	0.610
B: A + artist reach (followers) ← main model	41	10.396	7.696	0.680
C: B + artist popularity (LEAKY)	43	9.968	7.378	0.705



Next steps: Generate code with `ablation_df` New interactive sheet

Reading the ablation

- Going from **A → B** shows how much *artist reach* adds on top of the song itself.
- Going from **B → C** shows the inflation caused by the leaky `artist_popularity` feature. This gain is *not* genuine predictive power for a new, unreleased track, which is why the main model (B) excludes it.

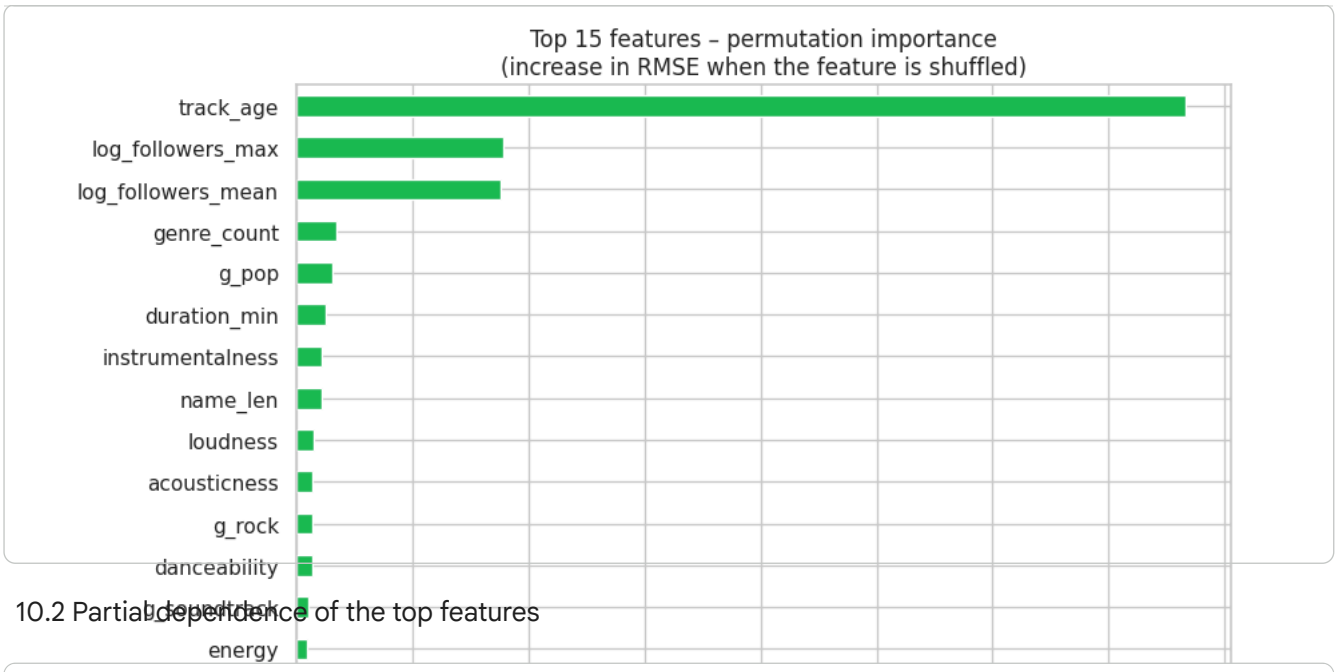
10. Interpretation of Results

10.1 Permutation importance (main model, test set)

```
final_model = best_hgb.fit(X_train[FEATURES_B], y_train)

pi_sample = X_test.sample(n=min(15000, len(X_test)), random_state=RANDOM_STATE)
pi = permutation_importance(final_model, pi_sample[FEATURES_B], pi_sample[TARGET],
                            n_repeats=5, random_state=RANDOM_STATE, n_jobs=-1,
                            scoring="neg_root_mean_squared_error")
imp = pd.Series(pi.importances_mean, index=FEATURES_B).sort_values(ascending=False)

imp.head(15)[::-1].plot.barh(figsize=(9, 6), color="#1DB954")
plt.title("Top 15 features – permutation importance\n(increase in RMSE when the feature is shuffled)")
plt.xlabel("Δ RMSE"); plt.tight_layout(); plt.show()
display(imp.head(10).round(3).to_frame("Δ RMSE"))
```



10.2 Partial dependence of the top features

```
top3 = imp.index[:3].tolist()
pd_sample = X_train[FEATURES_B].sample(n=min(8000, len(X_train)), random_state=RANDOM_STATE)
fig, ax = plt.subplots(1, 3, figsize=(17, 4.5))
PartialDependenceDisplay.from_estimator(final_model, pd_sample, top3, ax=ax, line_kw={"color": "#1DB954"})
plt.suptitle("Partial dependence – effect of top features on predicted popularity", y=1.03)
plt.tight_layout(); plt.show()
```

